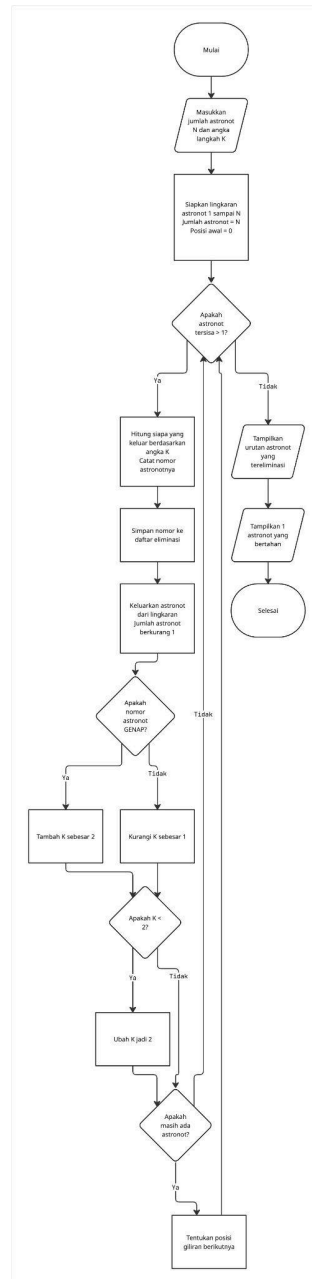


Pemrograman Dasar TLS 2026

Mirza Mahardika Widhiatna
Tourmaline

Problem 1 (The Last Astronaut)

Flowchart



Penjelasan logika sederhana

1. Persiapan:
Masukkan jumlah astronot (N) dan angka langkah awal (K). Buat barisan atau lingkaran berisi astronot dari nomor 1 sampai N.
2. Pilih Astronot yang Keluar:
Hitung langkah sebanyak K dari posisi saat ini untuk menentukan astronot mana yang gugur/tereliminasi.
3. Keluarkan dan Catat:
Hapus astronot tersebut dari lingkaran, lalu simpan nomornya ke dalam daftar urutan eliminasi.
4. Ubah Angka Langkah (K):
 - Jika nomor astronot yang keluar adalah angka genap, tambahkan nilai K sebanyak 2 ($K = K + 2$).
 - Jika nomor astronot yang keluar adalah angka ganjil, kurangi nilai K sebanyak 1 ($K = K - 1$).
 - Jika nilai K hasil perubahan tadi kurang dari 2, ubah kembali nilai K menjadi 2.
5. Ulangi Proses:
Tentukan posisi giliran untuk hitungan selanjutnya. Ulangi terus langkah 2 sampai 4 selama astronot yang tersisa di dalam lingkaran masih lebih dari 1 orang.
6. Selesai:
Jika hanya tersisa 1 astronot, tampilkan urutan astronot yang tereliminasi dan sebutkan 1 astronot terakhir yang berhasil bertahan.

Problem 2 (Alien-In-The-Middle)

Pseudocode

None

```
CONSTANT MAX = 1000
```

```
FUNCTION hitungPanjang(str: ARRAY OF CHAR) -> INTEGER
```

```
DECLARE i: INTEGER
```

```
BEGIN
```

```
    i = 0
```

```
    WHILE str[i] != '\0' DO
```

```
        i = i + 1
```

```
    ENDWHILE
```

```
    RETURN i
```

```
END
```

```
FUNCTION hurufKePosisi(huruf: CHAR) -> INTEGER
```

```
BEGIN
```

```
    RETURN (ASCII(huruf) - ASCII('A') + 1)
```

```
END
```

```
FUNCTION posisiKeHuruf(posisi: INTEGER) -> CHAR
```

```
BEGIN
```

```
    RETURN CHAR(ASCII('A') + posisi - 1)
```

```
END
```

```
PROCEDURE enkripsi(pesan: ARRAY OF CHAR, hasil: ARRAY OF CHAR)
```

```
DECLARE n, i, nilai: INTEGER
```

```
DECLARE posisiAsli, posisiBaru: ARRAY[MAX] OF INTEGER
```

```
BEGIN
```

```
    n = hitungPanjang(pesan)
```

```
    FOR i = 0 TO n - 1 DO
```

```
        posisiAsli[i] = hurufKePosisi(pesan[i])
```

```
    ENDFOR
```

```
    posisiBaru[0] = posisiAsli[0]
```

```
    FOR i = 1 TO n - 1 DO
```

```
        nilai = posisiAsli[i] + posisiAsli[i - 1]
```

```
        IF nilai > 26 THEN
```

```
            nilai = nilai - 26
```

```
        ENDIF
```

```
        posisiBaru[i] = nilai
```

```

    ENDFOR

    FOR i = 0 TO n - 1 DO
        hasil[i] = posisiKeHuruf(posisiBaru[i])
    ENDFOR
    hasil[n] = '\0'
END

PROCEDURE dekripsi(pesan: ARRAY OF CHAR, hasil: ARRAY OF CHAR)
DECLARE n, i, nilai: INTEGER
DECLARE posisiSandi, posisiAsli: ARRAY[MAX] OF INTEGER
BEGIN
    n = hitungPanjang(pesan)

    FOR i = 0 TO n - 1 DO
        posisiSandi[i] = hurufKePosisi(pesan[i])
    ENDFOR

    posisiAsli[0] = posisiSandi[0]

    FOR i = 1 TO n - 1 DO
        nilai = posisiSandi[i] - posisiAsli[i - 1]
        IF nilai <= 0 THEN
            nilai = nilai + 26
        ENDIF
        posisiAsli[i] = nilai
    ENDFOR

    FOR i = 0 TO n - 1 DO
        hasil[i] = posisiKeHuruf(posisiAsli[i])
    ENDFOR
    hasil[n] = '\0'
END

PROGRAM Main
DECLARE pesan, sandi, pesanAsli: ARRAY[MAX] OF CHAR
BEGIN
    PRINT "Masukkan pesan asli (huruf kapital, tanpa spasi): "
    READ pesan

```

```
CALL enkripsi(pesan, sandi)
PRINT "Pesan tersandi: ", sandi

CALL dekripsi(sandi, pesanAsli)
PRINT "Pesan asli: ", pesanAsli
END
```

Penjelasan logika sederhana

1. Konversi Huruf ke Angka:

Setiap huruf diubah menjadi urutan angka dalam alfabet (misalnya **A** menjadi 1, **B** menjadi 2, hingga **Z** menjadi 26).

2. Proses Enkripsi (Mengunci Pesan):

Huruf pertama dibiarkan tetap (tidak berubah). Huruf kedua dan seterusnya diubah dengan cara menambahkan nilai huruf tersebut dengan nilai huruf asli di depannya. Jika hasil penjumlahannya lebih dari 26, hasilnya dikurangi 26 agar nilainya kembali berputar di dalam rentang alfabet (1–26). Angka-angka ini kemudian diubah kembali menjadi huruf sandi.

3. Proses Dekripsi (Membuka Pesan):

Huruf pertama langsung diambil karena nilainya tidak pernah berubah. Huruf kedua dan seterusnya dikembalikan ke pesan asli dengan cara mengurangi nilai huruf sandi dengan nilai huruf asli sebelum dirinya. Jika hasilnya kurang dari atau sama dengan 0, nilainya ditambah 26 agar kembali ke posisi alfabet yang benar. Angka asli ini lalu diubah kembali menjadi huruf semula.